

Relazione Homework 2

Gruppo di lavoro:

Roberta Vallelunga (Matricola: 1000003065)

Antonio Zarbo (Matricola: 1000081002)

Simone Squillaci (Matricola: 1000081013)

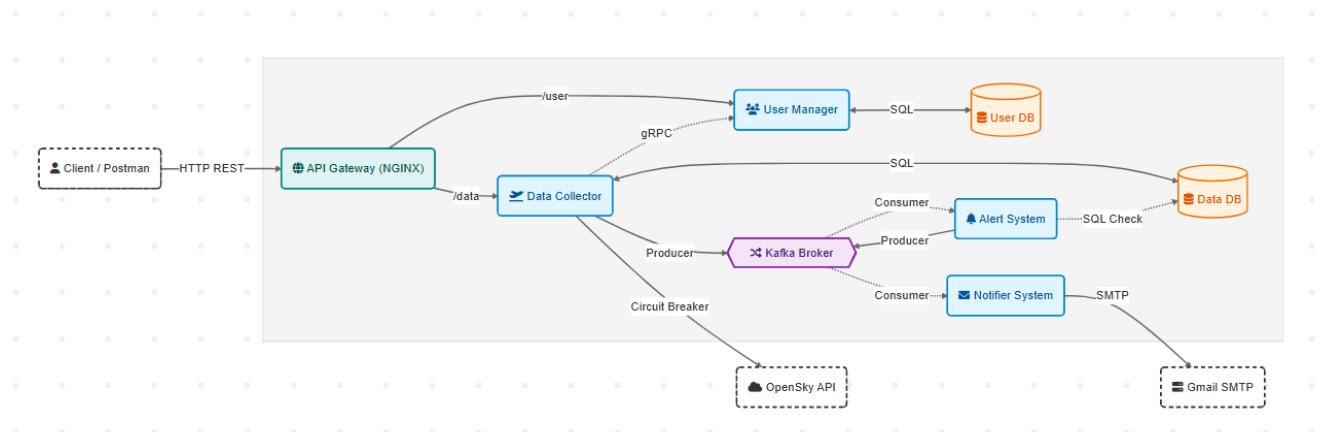
1. Descrizione del progetto

L'obiettivo del secondo homework è l'evoluzione del sistema a microservizi sviluppato nel precedente assignment, con un focus specifico sulla comunicazione asincrona, la resilienza e la sicurezza dell'accesso. Le principali estensioni architetturali introdotte includono:

- Un meccanismo di Circuit Breaker per proteggere le interazioni con il servizio esterno Open Sky Network.
- Un sistema di notifiche asincrone tramite e-mail basato sul superamento di soglie di volo personalizzabili dall'utente.
- L'introduzione di un API Gateway (NGINX) come punto unico di accesso al sistema.
- L'aggiunta di un Message Broker (Kafka) per disaccoppiare la fase di raccolta dati da quella di notifica.

2. Architettura

Il sistema conserva la suddivisione in servizi indipendenti e isolati all'interno di un ambiente Docker. L'uso di Docker Compose continua a garantire l'orchestrazione dell'intera infrastruttura, che è stata ampliata integrando i nuovi componenti sotto forma di container aggiuntivi.



Schema architetturale del sistema

2.1 Componenti Principali

API Gateway (NGINX): Agisce come unico punto di ingresso per il client (Postman). Instrada le richieste verso UserManager o DataCollector in base all'endpoint, proteggendo le porte dei servizi interni che non sono più esposte direttamente.

DataCollector (Producer): Oltre alle responsabilità originali (gRPC client verso User Manager e scraping dati), integra ora la logica del *Circuit Breaker*. Al termine dello scraping (recupero periodico dei dati di volo), agisce come *Kafka Producer*, inviando un messaggio al broker Kafka sul topic to-alert-system.

Apache Kafka (Message Broker): Componente centrale dell'architettura asincrona, responsabile della gestione dei flussi di dati. Grazie al suo meccanismo di log distribuito, assicura la persistenza dei messaggi e il disaccoppiamento tra *Producer* e *Consumer*, garantendo tolleranza ai guasti e affidabilità anche in condizioni di carico elevato.

AlertSystem (Consumer/Producer): Componente che consuma i messaggi dal topic “to-alert-system”, verifica se i dati aggiornati superano le soglie (high-value o low-value) impostate dagli utenti e, in caso positivo, produce un messaggio sul topic “to-notifier”, indicando l'utente e la condizione superata.

AlertNotifierSystem (Consumer): Componente che consuma i messaggi dal topic “to-notifier”. Alla ricezione di un evento, invia una e-mail all'utente tramite server SMTP (Gmail), specificando l'aeroporto e la condizione di soglia superata.

3. Scelte progettuali

Gestione dei Dati (Database)

Il database “dataDB” è stato aggiornato per supportare le nuove funzionalità.

Nella tabella interests, relativa alle preferenze degli utenti, sono state aggiunte le colonne “highValue” (Soglia superiore) e “lowValue” (Soglia inferiore). È stato implementato un vincolo di integrità per assicurare che, se entrambi presenti, highValue sia maggiore di lowValue.

Comunicazione Asincrona (Kafka)

L'adozione di Kafka è stata dettata dalla necessità di disaccoppiare nettamente la fase di raccolta dati (scraping) dalla logica di business relativa alle notifiche. L'architettura prevede due topic principali:

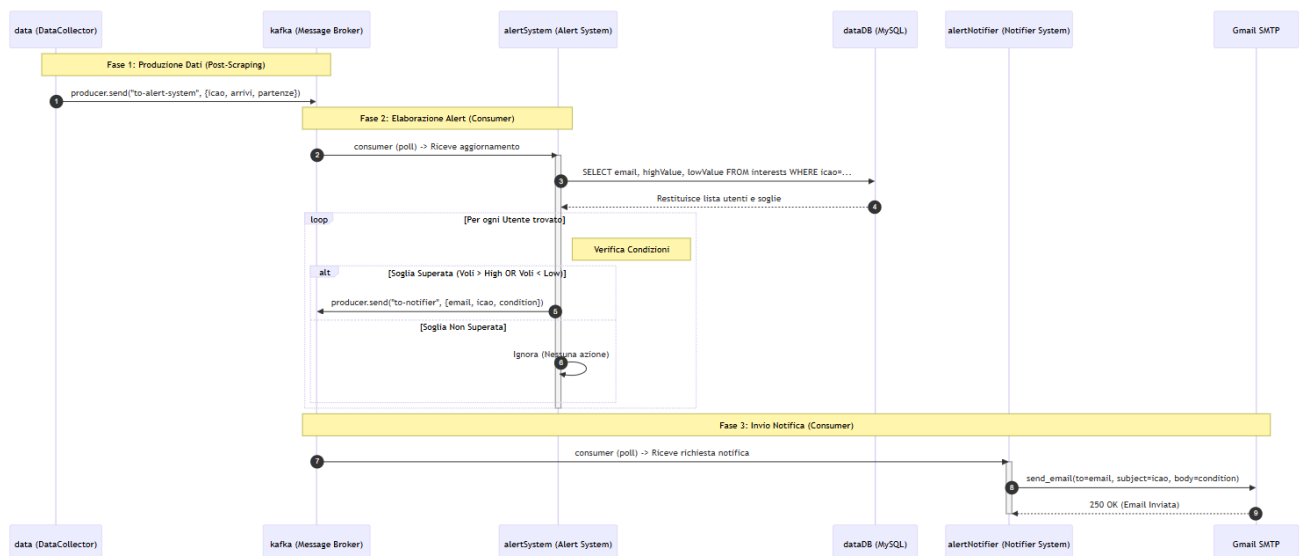
- **to-alert-system:** Canale dedicato ai dati operativi. Trasporta le informazioni grezze sui voli (arrivi e partenze) appena recuperate.
- **to-notifier:** Canale dedicato agli eventi di notifica. Contiene il payload finale <utente, aeroporto, condizione> pronto per essere processato dal servizio di invio e-mail.

- Per quanto riguarda il topic to-alert-system, il DataCollector è stato configurato per pubblicare un messaggio distinto per ogni singolo aeroporto elaborato, anziché aggregare tutti i dati in un unico payload massivo. Questa strategia di frammentazione offre due vantaggi cruciali: l'ottimizzazione delle performance, in quanto evita i rallentamenti tipici della serializzazione e deserializzazione di messaggi di grandi dimensioni, e la tolleranza ai guasti, in quanto, in caso di crash improvviso del DataCollector durante il ciclo di aggiornamento, si perde eventualmente solo il dato dell'aeroporto corrente e non l'intero dataset elaborato fino a quel momento.

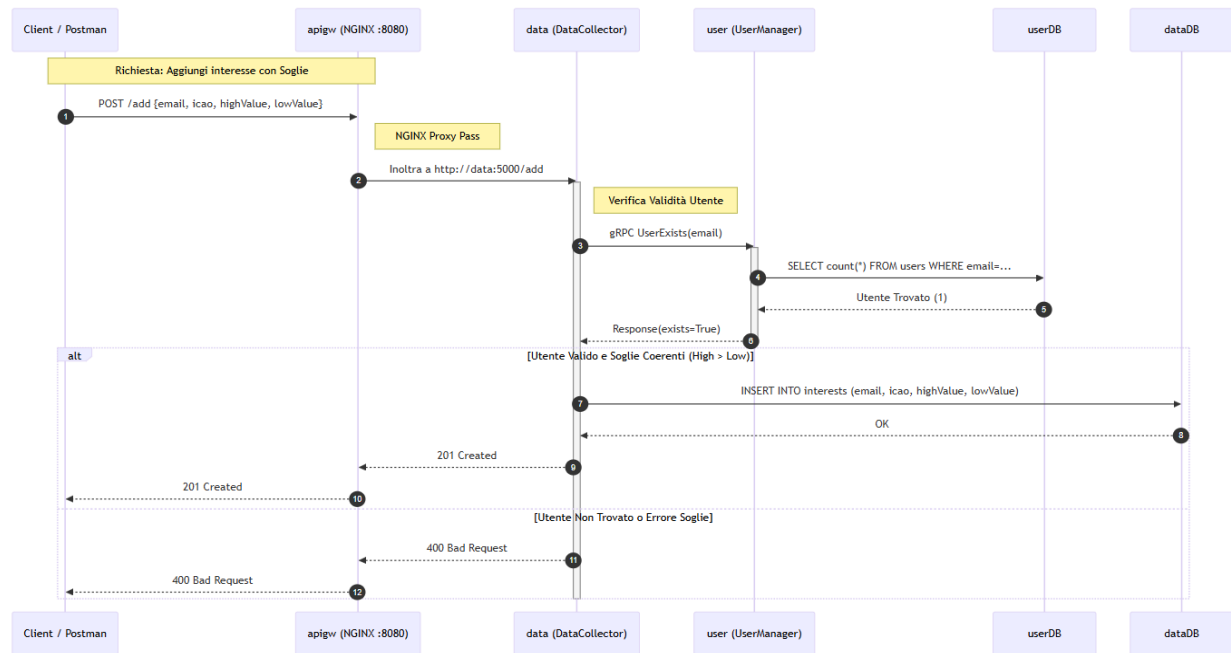
Resilienza (Circuit Breaker)

Per mitigare i rischi legati all'instabilità delle API di OpenSky, è stato implementato un *Circuit Breaker*. Questo componente previene il "cascade failure" (fallimento a cascata): se l'API esterna fallisce ripetutamente, il circuito si "apre" (stato OPEN) bloccando istantaneamente le richieste successive e permettendo al sistema di recuperare senza andare in crash.

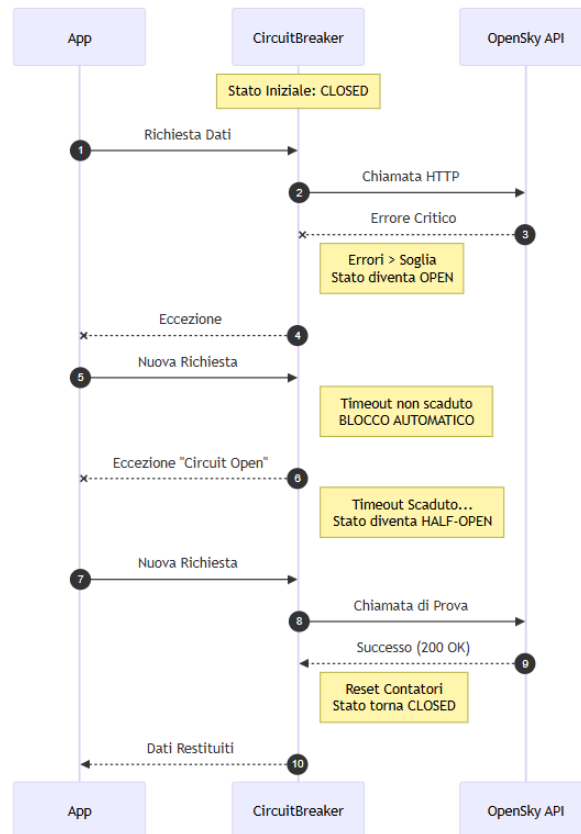
4. Diagrammi



Il flusso ha inizio con il **DataCollector** che pubblica i dati aggiornati sul Broker Kafka. Il messaggio viene consumato dall'**AlertSystem**, il quale interroga il database per identificare gli utenti interessati a quello specifico aeroporto e recuperare le relative soglie personalizzate. Successivamente avviene la valutazione delle condizioni: se le soglie non vengono superate, il processo termina senza ulteriori azioni. In caso contrario (violazione della soglia), l'AlertSystem produce un nuovo evento indirizzato all'**AlertNotifier**. Quest'ultimo componente consuma il messaggio e finalizza l'operazione inviando l'e-mail tramite il server SMTP di Gmail, da cui riceve la conferma di avvenuta spedizione.



Il diagramma evidenzia il ruolo centrale dell'**API Gateway (NGINX)** come unico punto di ingresso al sistema. Il client non interagisce più direttamente con i microservizi interni, ma indirizza le richieste esclusivamente alla porta **8080** esposta dal proxy, che provvede all'instradamento verso il backend. Nello specifico, lo schema illustra il flusso della chiamata `/add`, utilizzata per registrare un nuovo interesse di monitoraggio per un utente già registrato.



Il diagramma descrive la logica a stati finiti del Circuit Breaker. Nello stato iniziale **CLOSED**, il traffico fluisce regolarmente. Tuttavia, qualora il numero di errori consecutivi superi la soglia prefissata, avviene la transizione allo stato **OPEN**: in questa fase, il componente inibisce preventivamente ogni richiesta verso l'API esterna per evitarne il sovraccarico (*Fail Fast*). Scaduto il timeout di recupero, il sistema evolve in **HALF-OPEN**, consentendo l'esecuzione di una singola 'richiesta sonda'. L'esito di tale tentativo è determinante: in caso di successo, il circuito viene ripristinato (**CLOSED**); in caso di fallimento, torna immediatamente nello stato di blocco (**OPEN**), riavviando il timer di sicurezza.

5. API

L'interfaccia REST esposta dal *DataCollector* è stata ampliata per soddisfare i nuovi requisiti. L'endpoint di registrazione interessi (/add) è stato **esteso** per supportare, oltre ai dati standard, anche i parametri opzionali relativi alle soglie di allarme (*High Value* e *Low Value*). Parallelamente, è stato introdotto un **nuovo endpoint** dedicato alla gestione successiva delle preferenze, la cui specifica è riportata di seguito:

Metodo	Endpoint	Descrizione
POST	/modify_preference	Permette all'utente di modificare le soglie highValue e lowValue per un aeroporto monitorato. Nota: Inviando il valore 0 , la relativa soglia viene disattivata e ignorata dalle verifiche successive dell'Alert System.